

CIS Master Index & Roadmap

Your complete navigation guide — read this first

 All 21 Days

What This Package Is

A complete, production-grade implementation guide for the Customer Integration Simulator — a webhook processing platform with failure injection, retry orchestration, real-time observability, and AI-powered root cause analysis. Every command, every file, every line of code.

Package Contents

#	Filename	Contents	Days	Hours
0	00-MASTER-INDEX.docx	This file — navigation, roadmap, setup	—	30 min
1	01-FOUNDATION.docx	Tools, project structure, DB schema, env, models	1–3	8–10 hrs
2	02-BACKEND-CORE.docx	Event processor, API, failure engine, retry engine	4–7	12–15 hrs
3	03-FRONTEND.docx	React setup, components, event list, modal	8–12	10–12 hrs
4	04-FRONTEND-COMPLETE.docx	Dashboard assembly, simulator controls, App.js	13–14	6–8 hrs
5	05-AI-INTEGRATION.docx	Claude API, AI service, debugger UI	15–17	6–8 hrs
6	06-DEPLOYMENT.docx	Railway, Vercel, domain, portfolio, resume	18–21	8–10 hrs
S	SUPPLEMENT.docx	Missing files: constants.js, formatters.js, CORS, RLS	All	Add as you go

~150 pages
of content

21 days
to production

\$0 cost
all free tiers

1 live project
job-ready

The 21-Day Roadmap

Week	Days	Daily Goals
Week 1	Days 1–7	Backend Foundation
Week 2	Days 8–14	Frontend Build
Week 3	Days 15–21	AI + Deploy + Portfolio

Day	File to Open	What You Build
1	01-FOUNDATION.docx	Install tools, create project structure, Supabase account
2	01-FOUNDATION.docx	Database schema, environment variables, .gitignore
3	01-FOUNDATION.docx	Domain models, enums, database repository
4–5	02-BACKEND-CORE.docx	Event processor, FastAPI app, webhook endpoint
6–7	02-BACKEND-CORE.docx	Failure engine, retry engine, test all endpoints
8–9	03-FRONTEND.docx	Create React app, install deps, Tailwind, API client
10–12	03-FRONTEND.docx	Stats cards, event list, event detail modal, hooks
13–14	04-FRONTEND-COMPLETE.docx	Simulator controls, dashboard assembly, App.js
15–16	05-AI-INTEGRATION.docx	Claude API service, failure analysis endpoint
17	05-AI-INTEGRATION.docx	AI debugger UI component, test end-to-end
18–19	06-DEPLOYMENT.docx	Deploy backend to Railway, frontend to Vercel
20	06-DEPLOYMENT.docx	README, portfolio article, architecture diagram
21	06-DEPLOYMENT.docx	Final checklist, update portfolio, LinkedIn post

Before You Start — Prerequisites

Accounts (All Free, No Credit Card)

- ☐ GitHub — github.com
- ☐ Supabase — supabase.com (your PostgreSQL database)
- ☐ Railway — railway.app (backend hosting)
- ☐ Vercel — vercel.com (frontend hosting)
- ☐ Anthropic — console.anthropic.com (Claude API key — \$5 free credit included)

Tools to Install

```
python3 --version    # Need 3.11+
node --version       # Need 18+
npm --version        # Comes with Node
git --version        # Any recent version
```

Knowledge Required

Skill	Minimum Level
Python	Functions, classes, basic async/await
React	Components, props, useState/useEffect hooks
SQL	SELECT, INSERT, CREATE TABLE basics
Terminal	Comfortable running commands and navigating directories
Git	init, add, commit, push



Time Commitment

40–50 hours across 3 weeks. Plan 12–15 hours per week. Block calendar time before Day 1.



Golden Rule

Copy-paste code exactly first. Only customize once it works. This is a production-grade system — get it running before making it yours.

How to Use This Package

Scenario	Approach
Building from scratch (recommended)	Open this index, open the day's doc, follow step by step

You have parts already built	Jump directly to the relevant doc
Something isn't working	Open SUPPLEMENT.docx first — it fixes the most common issues
Ready to deploy	Open 06-DEPLOYMENT.docx
Done — need portfolio assets	Skip to the Portfolio section in 06-DEPLOYMENT.docx

About the Supplement

SUPPLEMENT.docx fills in files that were missing from the original guides: constants.js, formatters.js, the complete log repository method, CORS configuration, and local testing workflow. Check it when you hit import errors or CORS issues.

Success Criteria

After Day 7 you will have:

- Working backend API with all endpoints
- Webhook ingestion with state machine (9 states)
- Failure injection engine with configurable probabilities
- Exponential backoff retry engine (3 retries, 1s/2s/4s)
- Structured logging on every state transition

After Day 14 you will have:

- React dashboard at localhost:3000
- Real-time event list (auto-refreshes every 5 seconds)
- Event detail modal with full log history
- Webhook simulator with integration/event type selectors

After Day 21 you will have:

- Live production URL (Railway + Vercel)
- AI-powered failure analysis using Claude
- GitHub repo, README, portfolio article
- Resume bullets and LinkedIn post ready
- A project that demonstrates systems thinking to hiring managers

Start with 01-FOUNDATION.docx on Day 1 →

PART 1 Foundation

Tools · Project Structure · Database · Domain Models

 Days 1–3 · 01-FOUNDATION.docx

This Document Covers

Tool installation, project folder structure, Supabase database setup, .env configuration, domain models, and database repository. Complete all of this before opening Part 2.

Day 1 — Setup & Structure

Step 1: Install & Verify Tools

```
# Verify each tool after installing:
python3 --version      # Must be 3.11+
node --version         # Must be 18+
git --version          # Any version

# Mac installs (if missing):
brew install python@3.11
brew install node
brew install git

# Ubuntu installs:
sudo apt install python3.11 python3.11-venv nodejs npm git
```

Step 2: Create Project & Git

```
cd ~/Desktop    # or wherever you keep projects
mkdir customer-integration-simulator
cd customer-integration-simulator
git init
```

Step 3: Create .gitignore

```
cat > .gitignore << 'EOF'
# Python
__pycache__/
*.pyc
venv/
env/

# Environment — NEVER commit these
.env
.env.local
*.env

# Node
```

```
node_modules/  
build/  
dist/  
  
# IDE  
.vscode/  
.idea/  
.DS_Store  
  
# Logs  
*.log  
logs/  
EOF
```

Step 4: Create Full Folder Structure

```
# Backend  
mkdir -p  
backend/{app,core,domain/{models,enums,schemas},simulator,webhooks,infrastructure,database,services,api/v1/endpoints,tests}  
  
# Frontend  
mkdir -p  
frontend/src/{components/{Dashboard,Events,Simulator,AIDebugger},hooks,api,utils}  
  
# Docs  
mkdir docs  
  
# Verify - should show full tree  
find . -type d | sort
```

Step 5: Python Environment

```
cd backend  
python -m venv venv  
source venv/bin/activate # Windows: venv\Scripts\activate  
# You should see (venv) in your prompt
```

Step 6: Create requirements.txt

```
# backend/requirements.txt  
  
# Core Framework  
fastapi==0.104.1  
uvicorn[standard]==0.24.0  
pydantic==2.5.0  
  
# Database  
psycopg2-binary==2.9.9  
sqlalchemy==2.0.23  
  
# Environment  
python-dotenv==1.0.0
```

```
# HTTP & CORS
httpx==0.25.2

# AI Integration
anthropic==0.7.8

# Utilities
python-multipart==0.0.6
```

```
pip install -r requirements.txt
pip list      # Verify packages installed
```

Day 1 — Database Setup (Supabase)

Step 7: Create Supabase Project

- Go to supabase.com/dashboard → New project
- Name: customer-integration-simulator
- Generate a strong database password — SAVE IT NOW
- Choose region closest to you
- Free tier — no card needed
- Wait ~2 minutes for provisioning

Step 8: Get Connection String

- Dashboard → Settings (bottom left) → Database
- Scroll to Connection string → copy the URI format
- Format: postgresql://postgres:[PASSWORD]@db.xxx.supabase.co:5432/postgres
- Save this — you need it in .env

Step 9: Run Database Schema

Create `backend/database/schema.sql`, then paste the entire contents into Supabase → SQL Editor → New Query → Run:

```
-- Drop + recreate (safe for fresh start)
DROP TABLE IF EXISTS event_logs CASCADE;
DROP TABLE IF EXISTS events CASCADE;
DROP TABLE IF EXISTS integration_configs CASCADE;

CREATE TABLE events (
  event_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  integration_type   VARCHAR(50) NOT NULL,
  event_type        VARCHAR(100) NOT NULL,
  payload           JSONB NOT NULL,
  status            VARCHAR(50) NOT NULL,
  retry_count       INTEGER DEFAULT 0,
  max_retries       INTEGER DEFAULT 3,
  next_retry_at     TIMESTAMP,
```



```

failure_type          VARCHAR(50),
failure_reason        TEXT,
correlation_id        VARCHAR(100),
idempotency_key       VARCHAR(100) UNIQUE,
source_ip             VARCHAR(45),
created_at            TIMESTAMP DEFAULT NOW(),
updated_at            TIMESTAMP DEFAULT NOW(),
processed_at          TIMESTAMP,
processing_duration_ms INTEGER
);

CREATE INDEX idx_events_status ON events(status);
CREATE INDEX idx_events_integration ON events(integration_type);
CREATE INDEX idx_events_created ON events(created_at DESC);
CREATE INDEX idx_events_idempotency ON events(idempotency_key) WHERE
idempotency_key IS NOT NULL;

CREATE TABLE event_logs (
  log_id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id            UUID NOT NULL REFERENCES events(event_id) ON DELETE CASCADE,
  timestamp           TIMESTAMP DEFAULT NOW(),
  level              VARCHAR(20) NOT NULL,
  message             TEXT NOT NULL,
  metadata            JSONB,
  exception_type      VARCHAR(100),
  stack_trace         TEXT,
  duration_ms         INTEGER
);

CREATE INDEX idx_logs_event_id ON event_logs(event_id);
CREATE INDEX idx_logs_timestamp ON event_logs(timestamp DESC);

CREATE TABLE integration_configs (
  config_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  integration_type     VARCHAR(50) UNIQUE NOT NULL,
  enabled             BOOLEAN DEFAULT TRUE,
  failure_rate         DECIMAL(3,2) DEFAULT 0.20,
  avg_latency_ms       INTEGER DEFAULT 100,
  timeout_ms          INTEGER DEFAULT 5000,
  max_retries          INTEGER DEFAULT 3,
  created_at          TIMESTAMP DEFAULT NOW()
);

INSERT INTO integration_configs (integration_type, failure_rate, avg_latency_ms)
VALUES
('stripe', 0.15, 80), ('shopify', 0.20, 120), ('crm', 0.25, 150), ('generic',
0.20, 100);

-- Verify -- should see 4 rows:
SELECT * FROM integration_configs;

```

Day 2 — Environment & Config

Step 10: Create backend/.env

```

# backend/.env
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@db.xxx.supabase.co:5432/postgres
HOST=0.0.0.0
PORT=8000
ENVIRONMENT=development

```

```
FRONTEND_URL=http://localhost:3000
ANTHROPIC_API_KEY=sk-ant-api03-your-key-here
MAX_RETRIES=3
INITIAL_BACKOFF_MS=1000
BACKOFF_MULTIPLIER=2.0
DEFAULT_FAILURE_RATE=0.20
DEFAULT_LATENCY_MS=100
```

⚠ Never Commit .env

It's in .gitignore. Create a .env.example with placeholder values for the repo. Never put real keys in version control.

Day 3 — Domain Models

Step 11: Create Enums — backend/domain/enums/event_status.py

```
from enum import Enum

class EventStatus(str, Enum):
    RECEIVED = 'received'
    VALIDATED = 'validated'
    QUEUED = 'queued'
    PROCESSING = 'processing'
    SUCCESS = 'success'
    FAILED = 'failed'
    RETRYING = 'retrying'
    PERMANENT_FAILURE = 'permanent_failure'
    REJECTED = 'rejected'
    TIMED_OUT = 'timed_out'

class FailureType(str, Enum):
    TIMEOUT = 'timeout'
    NETWORK_ERROR = 'network_error'
    VALIDATION_ERROR = 'validation_error'
    AUTH_ERROR = 'auth_error'
    RATE_LIMIT = 'rate_limit'
    SERVER_ERROR = 'server_error'
    UNKNOWN = 'unknown'

class LogLevel(str, Enum):
    DEBUG = 'debug'
    INFO = 'info'
    WARNING = 'warning'
    ERROR = 'error'
```

State Machine Reference

State	Transitions From	Terminal?
RECEIVED	— (initial)	No
VALIDATED	RECEIVED	No

QUEUED	VALIDATED	No
PROCESSING	QUEUED / RETRYING	No
SUCCESS	PROCESSING	✅ Yes
FAILED	PROCESSING	No — retryable
RETRYING	FAILED / TIMED_OUT	No
PERMANENT_FAILURE	RETRYING (max reached)	✅ Yes
REJECTED	RECEIVED	✅ Yes
TIMED_OUT	PROCESSING	No — retryable

Step 12: Verify Models

```
cd backend && source venv/bin/activate
python -c "from domain.enums import EventStatus; print('✓ Enums OK')"
```

```
python -c "from domain.models import Event; print('✓ Models OK')"
```

✓ Days 1–3 Checklist



Stop Here

Do not open Part 2 until every item below is checked. Skipping will cause cascading failures.

Day 1

- ☐ python3 --version shows 3.11+
- ☐ node --version shows 18+
- ☐ git init ran without error
- ☐ .gitignore created
- ☐ All folders created (find . -type d | sort)
- ☐ (venv) shows in terminal prompt
- ☐ pip install -r requirements.txt completed

Day 2

- ☐ Supabase project created and provisioned
- ☐ Connection string copied and saved

- ☐ Schema SQL ran — no errors
- ☐ SELECT * FROM integration_configs shows 4 rows
- ☐ .env file created with real DATABASE_URL and ANTHROPIC_API_KEY

Day 3

- ☐ EventStatus, FailureType, LogLevel enums created
- ☐ Event and EventLog domain models created
- ☐ Database repository created with get/create/update methods
- ☐ python -c 'from domain.models import Event' runs without error
- ☐ Database connection test passes

Next: Open 02-BACKEND-CORE.docx on Day 4 →

PART 2 Backend Core

Event Processor · API · Failure Engine · Retry Engine

 Days 4–7 · 02-BACKEND-CORE.docx

This Document Covers

FastAPI application, CORS config, event processor, webhook endpoints, failure simulation engine, retry engine with exponential backoff, and API testing. Prerequisite: Part 1 complete.

Days 4–5 — FastAPI Application

main.py — app/main.py

This includes the **CORS fix** that prevents frontend connection errors (was missing in the original guide):

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware # CRITICAL — do not skip
from app.config import settings
from api.v1.router import api_router
from database.connection import db_manager
import logging

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')

app = FastAPI(
    title='Customer Integration Simulator',
    description='Webhook integration testing platform',
    version='1.0.0',
    docs_url='/api/docs',
    redoc_url='/api/redoc'
)

# — CORS — Required for frontend to reach backend —————
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        'http://localhost:3000',
        'http://localhost:3001',
        settings.FRONTEND_URL,
        'https://simulator.yourdomain.com',
    ],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)
# —————

@app.on_event('startup')
async def startup():
    print('🚀 Starting CIS')
    ok = db_manager.test_connection()
```

```

    print('✓ DB connected' if ok else '✗ DB FAILED')

@app.get('/health')
async def health():
    db = db_manager.test_connection()
    return {'status': 'healthy' if db else 'unhealthy', 'database': db}

app.include_router(api_router, prefix='/api/v1')

if __name__ == '__main__':
    import uvicorn
    uvicorn.run('app.main:app', host=settings.HOST,
                port=settings.PORT, reload=True)

```

API Endpoints — Complete Reference

Method	Path	Purpose
POST	/api/v1/webhook/{integration_type}	Ingest event — returns event_id
GET	/api/v1/events	List all events with status
GET	/api/v1/events/{id}	Full event detail + logs
GET	/api/v1/events/{id}/logs	State transition log history
POST	/api/v1/events/{id}/retry	Manually retry a failed event
GET	/api/v1/metrics	KPI aggregates for dashboard
GET	/health	Health check (no prefix)
GET	/api/docs	Interactive Swagger UI

Days 4–5 — Event Processor

Retry Policy

Parameter	Value / Behaviour
Max retries	3 (configurable via MAX_RETRIES env var)
Backoff formula	$\text{wait} = 2^{\text{retry_count}} \rightarrow 1\text{s}, 2\text{s}, 4\text{s}$
Retryable statuses	FAILED TIMED_OUT
Non-retryable	PERMANENT_FAILURE from processing (no retry)

Exhausted retries	Status → PERMANENT_FAILURE
-------------------	----------------------------

Required Log Fields — Every State Transition

- Timestamp (UTC ISO 8601)
- Previous state → New state
- Error type (if applicable)
- Retry count at time of transition
- Processing latency in milliseconds

Days 4–5 — Failure Engine

Failure Probability Defaults

All percentages configurable via `.env` variables:

Failure Mode	Default %	Example Scenarios
Success	50%	Processed cleanly, no issues
Temporary Failure	15%	DB timeout, rate limit, 503 response
Timeout	10%	Processing exceeded timeout threshold
High Latency	20%	Network delay 500ms–3000ms (may still succeed)
Permanent Failure	5%	Malformed payload, business rule violation

Days 6–7 — Retry Engine

Retry Logic Flow

```
# pseudo-code for retry_engine.py

def handle_failed_event(event):
    if event.status in [FAILED, TIMED_OUT]:
        if event.retry_count < event.max_retries:
            wait = 2 ** event.retry_count    # 1s, 2s, 4s
            event.retry_count += 1
            event.status = RETRYING
            schedule_retry(event, wait_seconds=wait)
        else:
            event.status = PERMANENT_FAILURE
            # No more retries – dead letter
    # Always log the transition
```

Days 6–7 — Test the Backend

Start Server

```
cd backend && source venv/bin/activate
python app/main.py
# Expected output:
# 🚀 Starting CIS
# ✓ DB connected
# INFO: Uvicorn running on http://0.0.0.0:8000
```

Smoke Tests (run in separate terminal)

```
# Health check
curl http://localhost:8000/health
# → {"status":"healthy","database":true}

# Send test webhook
curl -X POST http://localhost:8000/api/v1/webhook/stripe \
  -H 'Content-Type: application/json' \
  -d '{"event_type":"payment.succeeded","data":{"amount":5000}}'
# → {"event_id":"uuid...", "status":"received"}

# List events
curl http://localhost:8000/api/v1/events
# → [{"event_id":"...", "status":"success|failed|..."}]

# View API docs
# Open http://localhost:8000/api/docs in browser
```

✓ Days 4–7 Checklist

🛑 Stop Here

Do not open Part 3 until every item below is checked.

- ☐ (venv) is active when running commands
- ☐ python app/main.py starts without errors
- ☐ ✓ DB connected appears in startup logs
- ☐ GET /health returns {status: healthy}
- ☐ POST /webhook/stripe returns event_id
- ☐ Event appears in Supabase events table
- ☐ Retry loop visible in logs (watch for RETRYING state)
- ☐ GET /events returns JSON array
- ☐ /api/docs opens Swagger UI in browser
- ☐ CORS middleware added to main.py
- ☐ get_logs_for_event() method added to repository (see SUPPLEMENT.docx)

Next: Open 03-FRONTEND.docx on Day 8 →

PART 3 Frontend Dashboard

React · Tailwind · Components · Real-time Events

 Days 8–12 · 03-FRONTEND.docx

This Document Covers

React app setup, Tailwind CSS, API client, custom hooks, stats cards, event list, and event detail modal. Prerequisite: Part 2 backend running on :8000.

Days 8–9 — React Setup

Step 1: Create App

```
cd frontend
npx create-react-app .
npm install axios date-fns recharts @heroicons/react
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Step 2: Configure Tailwind — tailwind.config.js

```
module.exports = {
  content: ['./src/**/*.{js,jsx,ts,tsx}'],
  theme: { extend: {} },
  plugins: [],
};
```

Step 3: Add Tailwind to src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Step 4: Create frontend/.env

```
REACT_APP_API_URL=http://localhost:8000/api/v1
```

Step 5: API Client — src/api/client.js

```
import axios from 'axios';

const apiClient = axios.create({
```

```

    baseURL: process.env.REACT_APP_API_URL,
    headers: { 'Content-Type': 'application/json' },
    timeout: 10000,
  });

  apiClient.interceptors.response.use(
    response => response,
    error => {
      console.error('API Error:', error.response?.data || error.message);
      return Promise.reject(error);
    }
  );

  export default apiClient;

```

Step 6: Custom Hooks

```

// src/hooks/useEvents.js
import { useState, useEffect } from 'react';
import apiClient from '../api/client';

export const useEvents = (refreshInterval = 5000) => {
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchEvents = async () => {
      try {
        const { data } = await apiClient.get('/events');
        setEvents(data);
        setError(null);
      } catch (e) {
        setError(e.message);
      } finally {
        setLoading(false);
      }
    };

    fetchEvents();
    const interval = setInterval(fetchEvents, refreshInterval);
    return () => clearInterval(interval);
  }, [refreshInterval]);

  return { events, loading, error };
};

```

Days 10–12 — Dashboard Components

Component Map

Component	Responsibility
components/Dashboard/Dashboard.jsx	4 KPI stats cards grid

components/Dashboard/StatsCard.jsx	Individual stat: icon + value + subtitle
components/Events/EventList.jsx	Table of events with auto-refresh
components/Events/EventDetailModal.jsx	Full log history viewer on click
src/utils/constants.js	STATUS_COLORS, INTEGRATION_TYPES, EVENT_TYPES ← see SUPPLEMENT
src/utils/formatters.js	formatTimestamp, formatDuration, formatPercentage ← see SUPPLEMENT

Status Color Reference

Status	Tailwind Class	Meaning
success	bg-green-100 text-green-800	Completed OK
processing	bg-yellow-100 text-yellow-800	Running
retrying	bg-orange-100 text-orange-800	Backoff wait
failed / permanent_failure	bg-red-100 text-red-800	Failed
received / validated / queued	bg-blue-100 text-blue-800	In pipeline
rejected	bg-gray-100 text-gray-800	Validation failed
timed_out	bg-red-100 text-red-800	Exceeded timeout

✓ Days 8–12 Checklist



Stop Here

Do not open Part 4 until these pass.

- ☐ npm start runs — no errors in terminal
- ☐ http://localhost:3000 opens in browser
- ☐ No CORS errors in browser console
- ☐ Dashboard renders (no blank page)
- ☐ Stats cards load (even if all zeros)
- ☐ Events list renders
- ☐ Clicking an event opens the detail modal

- ☐ constants.js created (see SUPPLEMENT.docx)
- ☐ formatters.js created (see SUPPLEMENT.docx)
- ☐ No import errors in browser console

Next: Open 04-FRONTEND-COMPLETE.docx on Day 13 →

PART 4 Frontend Complete

Dashboard Assembly · Simulator Controls · App.js

 Days 13–14 · 04-FRONTEND-COMPLETE.docx

This Document Covers

Complete dashboard assembly, simulator controls with integration selector, App.js final assembly, and local end-to-end testing. Prerequisite: Part 3 components working.

Day 13 — Dashboard Assembly

Dashboard.jsx — components/Dashboard/Dashboard.jsx

```
import React from 'react';
import { ChartBarIcon, CheckCircleIcon, XCircleIcon, ClockIcon } from
 '@heroicons/react/24/outline';
import StatsCard from './StatsCard';
import { useStats } from '../../hooks/useStats';
import { formatPercentage } from '../../utils/formatters';

const Dashboard = () => {
  const { stats, loading } = useStats();
  if (loading) return (
    <div className='flex justify-center items-center h-64'>
      <div className='animate-spin rounded-full h-12 w-12 border-b-2 border-blue-
600'></div>
    </div>
  );
  return (
    <div className='grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6'>
      <StatsCard title='Total Events' value={stats?.total_events || 0}
icon={ChartBarIcon} color='blue' />
      <StatsCard title='Successful' value={stats?.successful || 0}
        subtitle={`$${formatPercentage(stats?.success_rate || 0)} rate`}
        icon={CheckCircleIcon} color='green' />
      <StatsCard title='Failed' value={stats?.failed || 0}
        subtitle={`$${stats?.permanent_failures || 0} permanent`}
        icon={XCircleIcon} color='red' />
      <StatsCard title='Processing' value={stats?.processing || 0}
        icon={ClockIcon} color='yellow' />
    </div>
  );
};
export default Dashboard;
```

SimulatorControls.jsx — components/Simulator/SimulatorControls.jsx

```
import React, { useState } from 'react';
import { sendWebhook } from '../../api/endpoints';
import { INTEGRATION_TYPES, EVENT_TYPES } from '../../utils/constants';
```

```

const SimulatorControls = ({ onEventCreated }) => {
  const [integration, setIntegration] = useState('stripe');
  const [eventType, setEventType] = useState('payment.succeeded');
  const [sending, setSending] = useState(false);
  const [result, setResult] = useState(null);

  const handleSend = async () => {
    setSending(true);
    setResult(null);
    try {
      const payload = {
        event_type: eventType,
        data: { amount: Math.floor(Math.random()*10000)+100 },
        idempotency_key: `${integration}_${Date.now()}`,
      };
      const res = await sendWebhook(integration, payload);
      setResult({ success: true, data: res });
      if (onEventCreated) onEventCreated(res);
    } catch (e) {
      setResult({ success: false, error: e.message });
    } finally { setSending(false); }
  };

  return (
    <div className='bg-white rounded-lg shadow p-6'>
      <h2 className='text-lg font-semibold mb-4'>Webhook Simulator</h2>
      <div className='space-y-4'>
        <select value={integration}
          onChange={e => { setIntegration(e.target.value);
            setEventType(EVENT_TYPES[e.target.value][0]); }}
          className='w-full px-3 py-2 border rounded-lg'>
          {INTEGRATION_TYPES.map(t => <option key={t.value}
value={t.value}>{t.label}</option>)}
        </select>
        <select value={eventType} onChange={e => setEventType(e.target.value)}
          className='w-full px-3 py-2 border rounded-lg'>
          {(EVENT_TYPES[integration]||[]).map(t => <option key={t}
value={t}>{t}</option>)}
        </select>
        <button onClick={handleSend} disabled={sending}
          className={`w-full py-3 rounded-lg font-medium text-white ${sending?'bg-
gray-400':'bg-blue-600 hover:bg-blue-700'}}>
          {sending ? 'Sending...' : 'Send Test Webhook'}
        </button>
        {result && (
          <div className={`p-4 rounded-lg ${result.success?'bg-green-50 border-
green-200':'bg-red-50 border-red-200'} border`}>
            {result.success
              ? <p className='text-green-800 text-sm'>✓ Sent — Event ID:
{result.data?.event_id}</p>
              : <p className='text-red-800 text-sm'>X Failed: {result.error}</p>
            }
          </div>
        )}
      </div>
    </div>
  );
};
export default SimulatorControls;

```

Day 14 — Final App.js Assembly

src/App.js — Complete File

```
import React from 'react';
import Dashboard from '../components/Dashboard/Dashboard';
import EventList from '../components/Events/EventList';
import SimulatorControls from '../components/Simulator/SimulatorControls';

function App() {
  return (
    <div className='min-h-screen bg-gray-50'>
      <header className='bg-white shadow-sm'>
        <div className='max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-4'>
          <h1 className='text-2xl font-bold text-gray-900'>
            Customer Integration Simulator
          </h1>
          <p className='mt-1 text-sm text-gray-500'>
            Webhook integration testing and debugging platform
          </p>
        </div>
      </header>
      <main className='max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8'>
        <div className='space-y-8'>
          <Dashboard />
          <div className='grid grid-cols-1 lg:grid-cols-3 gap-8'>
            <div className='lg:col-span-1'>
              <SimulatorControls />
            </div>
            <div className='lg:col-span-2'>
              <EventList />
            </div>
          </div>
        </div>
      </main>
      <footer className='bg-white border-t mt-12 py-6'>
        <p className='text-center text-sm text-gray-500'>
          Customer Integration Simulator © 2026
        </p>
      </footer>
    </div>
  );
}
export default App;
```

Day 14 — End-to-End Local Test

Run this full test before moving to AI integration. If anything fails, fix it now — not after deploying.

```
# Terminal 1 — Backend
cd backend && source venv/bin/activate && python app/main.py

# Terminal 2 — Frontend
cd frontend && npm start

# Manual test sequence in browser:
# 1. Visit http://localhost:3000
# 2. Click 'Send Test Webhook' → see event appear in list
# 3. Click an event → detail modal opens with logs
# 4. Watch stats cards update
# 5. Open browser DevTools → Console — no errors
```


✓ Days 13–14 Checklist

⛔ Stop Here

Do not open Part 5 until every item below passes.

- ☐ npm start runs — no errors
- ☐ Dashboard loads at `http://localhost:3000`
- ☐ Stats cards show real numbers from backend
- ☐ Event list auto-refreshes every 5 seconds
- ☐ Send Test Webhook → event appears in list
- ☐ Clicking event → detail modal with logs
- ☐ Status badges colored correctly
- ☐ Browser DevTools console — zero errors
- ☐ `constants.js` confirmed working (no import errors)
- ☐ `formatters.js` confirmed working (dates/numbers format correctly)

Next: Open 05-AI-INTEGRATION.docx on Day 15 →

PART 5 AI Integration

Claude API · Failure Analysis · AI Debugger UI

 Days 15–17 · 05-AI-INTEGRATION.docx

This Document Covers

AI service with Claude API, failure analysis endpoint, AI debugger React component, credit conservation tips, and debug mode for local testing. Prerequisite: Part 4 fully working.

Days 15–16 — Backend AI Service

How It Works

- User clicks 'Analyze Failure' on a failed event in the dashboard
- Frontend calls POST /api/v1/ai-debug/{event_id}
- Backend fetches event logs and sends them to Claude API
- Claude returns structured analysis: root cause, fix, prevention, code example
- Frontend displays the 4 sections in colour-coded panels

When It Activates

The button only appears when event status is failed, permanent_failure, or timed_out. It is hidden for success, processing, retrying, and received states.

AI Service — backend/services/ai_service.py

```
import os
from anthropic import Anthropic
import logging
from typing import Dict, Any, List

logger = logging.getLogger(__name__)

class AIService:
    def __init__(self):
        api_key = os.getenv('ANTHROPIC_API_KEY')
        self.debug_mode = os.getenv('ENVIRONMENT') == 'development'
        self.client = Anthropic(api_key=api_key) if api_key else None
        if not api_key:
            logger.warning('ANTHROPIC_API_KEY not set - AI disabled')

    async def analyze_failure(self, event_type, integration_type,
                             failure_type, failure_reason, logs, retry_count):
        # Debug mode - returns mock response (saves API credits)
        if self.debug_mode or not self.client:
            return {
                'root_cause': '[DEBUG] Simulated timeout - network unreachable',
```

```

        'immediate_fix': '- Check network connectivity\n- Increase
timeout',
        'prevention': '- Add circuit breaker\n- Monitor latency metrics',
        'code_example': 'timeout_ms = 10000 # Increase from default'
    }

    prompt = self._build_prompt(event_type, integration_type,
                                failure_type, failure_reason, logs,
retry_count)
    response = self.client.messages.create(
        model='claude-3-5-sonnet-20241022',
        max_tokens=1500,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return self._parse(response.content[0].text)

    def _build_prompt(self, event_type, integration, failure_type,
                      reason, logs, retries):
        return f'''You are an expert integration engineer analyzing a webhook
failure.

Event: {integration}.{event_type}
Failure: {failure_type} - {reason}
Retries: {retries}
Logs:
{logs}

Respond with EXACTLY these section headers:

## Root Cause
[2-3 sentences]

## Immediate Fix
- [action]

## Prevention Strategy
- [strategy]

## Code Example
```python
[code if applicable]
```'''

    def _parse(self, text):
        sections = {}
        {'root_cause': '', 'immediate_fix': '', 'prevention': '', 'code_example': ''}
        current = None
        for line in text.split('\n'):
            if '## Root Cause' in line: current = 'root_cause'
            elif '## Immediate Fix' in line: current = 'immediate_fix'
            elif '## Prevention Strategy' in line: current = 'prevention'
            elif '## Code Example' in line: current = 'code_example'
            elif current and line.strip(): sections[current] += line + '\n'
        return {k: v.strip() for k,v in sections.items()}

ai_service = AIService()

```

Days 15–16 — Debug Endpoint

[backend/api/v1/endpoints/ai_debug.py](#)

```

from fastapi import APIRouter, HTTPException
from uuid import UUID
from services.ai_service import ai_service
from database.repository import event_repository, log_repository

router = APIRouter()

@router.post('/{event_id}')
async def analyze_event_failure(event_id: UUID):
    event = event_repository.get_event(event_id)
    if not event:
        raise HTTPException(status_code=404, detail='Event not found')
    if event.status not in ['failed', 'permanent_failure', 'timed_out']:
        raise HTTPException(status_code=400, detail='Event has not failed')

    logs = log_repository.get_logs_for_event(event_id)
    logs_text = '\n'.join([f"[{l['timestamp']}] {l['level']}: {l['message']}" for
l in logs])

    analysis = await ai_service.analyze_failure(
        event_type=event.event_type,
        integration_type=event.integration_type,
        failure_type=str(event.failure_type),
        failure_reason=event.failure_reason or 'Unknown',
        logs=logs_text,
        retry_count=event.retry_count
    )
    return {'event_id': str(event_id), 'analysis': analysis}

```

Add to backend/api/v1/router.py:

```

from api.v1.endpoints import ai_debug
api_router.include_router(ai_debug.router, prefix='/ai-debug', tags=['ai-debug'])

```

Day 17 — AI Debugger UI

components/AIDebugger/AIDebugger.jsx

```

import React, { useState } from 'react';
import { SparklesIcon } from '@heroicons/react/24/outline';
import apiClient from '../../api/client';

const AIDebugger = ({ event }) => {
    const [analysis, setAnalysis] = useState(null);
    const [analyzing, setAnalyzing] = useState(false);
    const [error, setError] = useState(null);

    // Only show for failed events
    if (!['failed', 'permanent_failure', 'timed_out'].includes(event.status))
        return null;

    const analyze = async () => {
        setAnalyzing(true); setError(null); setAnalysis(null);
        try {
            const { data } = await apiClient.post(`/ai-debug/${event.event_id}`);
            setAnalysis(data.analysis);
        } catch (e) {

```

```

        setError(e.response?.data?.detail || 'Analysis failed');
    } finally { setAnalyzing(false); }
};

return (
    <div className='mt-6 border-t pt-6'>
        <div className='flex items-center justify-between mb-4'>
            <h3 className='text-lg font-semibold flex items-center'>
                <SparklesIcon className='h-5 w-5 mr-2 text-purple-600' />
                AI Debug Assistant
            </h3>
            <button onClick={analyze} disabled={analyzing}
                className={`px-4 py-2 rounded-lg font-medium text-white
                    ${analyzing?'bg-gray-400':'bg-purple-600 hover:bg-purple-700'}}>
                {analyzing ? 'Analyzing...' : 'Analyze Failure'}
            </button>
        </div>
        <div className='bg-red-50 border border-red-200 rounded p-4 text-red-800 text-sm'>{error}</div>
        {analyzing && <div className='flex justify-center py-8'><div
            className='animate-spin rounded-full h-12 w-12 border-b-2 border-purple-600'
            /></div>}
        {analysis && (
            <div className='space-y-4'>
                <div className='bg-red-50 border border-red-200 rounded p-4'>
                    <h4 className='font-semibold text-red-900 mb-2'>Root Cause</h4>
                    <p className='text-sm text-red-800 whitespace-pre-wrap'>{analysis.root_cause}</p>
                </div>
                <div className='bg-yellow-50 border border-yellow-200 rounded p-4'>
                    <h4 className='font-semibold text-yellow-900 mb-2'>Immediate Fix</h4>
                    <div className='text-sm text-yellow-800 whitespace-pre-wrap'>{analysis.immediate_fix}</div>
                </div>
                <div className='bg-blue-50 border border-blue-200 rounded p-4'>
                    <h4 className='font-semibold text-blue-900 mb-2'>Prevention
                    Strategy</h4>
                    <div className='text-sm text-blue-800 whitespace-pre-wrap'>{analysis.prevention}</div>
                </div>
                {analysis.code_example && (
                    <div className='bg-gray-50 border border-gray-200 rounded p-4'>
                        <h4 className='font-semibold text-gray-900 mb-2'>Code Example</h4>
                        <pre className='text-xs overflow-x-auto'>{analysis.code_example}</code></pre>
                    </div>
                )}
                <button onClick={() => setAnalysis(null)} className='text-sm text-gray-600 hover:text-gray-900'>
                    Clear Analysis
                </button>
            </div>
        )}
    </div>
);
};
export default AIDebugger;

```

Add to EventDetailModal.jsx at the bottom, before the closing tag:

```
import AIDebugger from '../AIDebugger/AIDebugger';
```

```
// Inside modal content, after the logs section:
{detailedEvent && <AIDebugger event={detailedEvent} />}
```

Credit Usage Guide

Scenario	Note
Each AI analysis call	~\$0.006 (approx 2,000 tokens)
\$5 free credit	~800 analyses total
Development tip	Set ENVIRONMENT=development — uses mock response, costs \$0
Testing tip	Set DEFAULT_FAILURE_RATE=0.80 to generate lots of failures cheaply
Production	Only charged when a real user clicks 'Analyze Failure'

✓ Days 15–17 Checklist

🛑 Stop Here

Do not open Part 6 until every item below passes.

- ☐ ANTHROPIC_API_KEY set in backend/.env
- ☐ AI service initializes — check startup logs for warnings
- ☐ ai_debug router added to api/v1/router.py
- ☐ POST /api/v1/ai-debug/{id} returns 200 on a failed event
- ☐ SparklesIcon button visible only on failed events
- ☐ Clicking 'Analyze Failure' shows loading spinner
- ☐ 4 analysis panels render with content
- ☐ Debug mode working (mock response in development)
- ☐ get_logs_for_event() added to repository (see SUPPLEMENT.docx)

Next: Open 06-DEPLOYMENT.docx on Day 18 →

PART 6 Deployment & Portfolio

[Railway](#) · [Vercel](#) · [Domain](#) · [README](#) · [Resume](#) · [LinkedIn](#)

 [Days 18–21](#) · [06-DEPLOYMENT.docx](#)

This Document Covers

Backend deployment to Railway, frontend deployment to Vercel, custom domain setup, local pre-deploy testing, README template, portfolio article, resume bullets, and LinkedIn post. Prerequisite: All parts 1–5 working locally.

Critical

Test everything locally on Day 17 before opening this doc. Do not attempt to debug deployment issues if local doesn't work.

Days 18–19 — Pre-Deploy Local Test

```
# Run this checklist BEFORE deploying anything

# Terminal 1
cd backend && source venv/bin/activate && python app/main.py

# Terminal 2
cd frontend && npm start

# Terminal 3 — smoke tests
curl http://localhost:8000/health
# → {"status":"healthy","database":true}

curl -X POST http://localhost:8000/api/v1/webhook/stripe \
  -H 'Content-Type: application/json' \
  -d '{"event_type":"payment.succeeded","data":{"amount":5000}}'
# → {"event_id":"uuid","status":"received"}
```

Local Pre-Deploy Checklist

- ☐ Backend runs on :8000 — health returns healthy
- ☐ Frontend runs on :3000 — no console errors
- ☐ Send Webhook → event appears in list
- ☐ Event detail modal opens with logs
- ☐ AI analysis returns result (or debug mock)
- ☐ No errors in backend terminal

Days 18–19 — Backend → Railway

Step 1: Prepare Backend Files

```
# backend/Procfile
web: uvicorn app.main:app --host 0.0.0.0 --port $PORT

# backend/runtime.txt
python-3.11.6

# Add to backend/requirements.txt:
gunicorn==21.2.0
```

Step 2: Deploy to Railway

```
npm install -g @railway/cli
railway login
cd backend && railway init

# Set all environment variables
railway variables set DATABASE_URL='postgresql://...'
railway variables set ANTHROPIC_API_KEY='sk-ant-...'
railway variables set ENVIRONMENT='production'
railway variables set MAX_RETRIES='3'
railway variables set DEFAULT_FAILURE_RATE='0.20'
# FRONTEND_URL - set AFTER you get Vercel URL

railway up

# Verify
curl https://your-app.up.railway.app/health
# → {"status":"healthy","database":true}
```

Days 18–19 — Frontend → Vercel

Step 3: Deploy to Vercel

```
npm install -g vercel
cd frontend && vercel

# Vercel will ask:
# Project name? → customer-integration-simulator
# Override settings? → No

# After deploy, go to Vercel dashboard → Settings → Environment Variables:
# REACT_APP_API_URL = https://your-backend.up.railway.app/api/v1

# Redeploy with the env var:
vercel --prod
```

Step 4: Update CORS & Redeploy Backend

```
# In backend/app/main.py, add your Vercel URL to allow_origins:
allow_origins=[
    'http://localhost:3000',
```



```
settings.FRONTEND_URL,  
'https://your-app.vercel.app',           # Add this  
'https://simulator.yourdomain.com',      # Add when domain ready  
],  
  
# Then redeploy backend:  
cd backend && railway up
```

Days 18–19 — Custom Domain (Vercel)

- Vercel Dashboard → your project → Settings → Domains
- Add: simulator.yourdomain.com
- Vercel shows DNS records — add them in your domain registrar
- Wait 5–30 minutes for DNS propagation
- SSL auto-provisioned by Vercel
- Update CORS in backend to include the new domain

Day 20 — Documentation

GitHub README Template

```
# Customer Integration Simulator  
  
Production-grade webhook integration testing platform with AI-powered  
failure analysis, retry orchestration, and real-time observability.  
  
## Features  
- Webhook ingestion from Stripe, Shopify, CRM integrations  
- Configurable failure simulation (timeout, network, validation errors)  
- Exponential backoff retry engine (1s, 2s, 4s — max 3 retries)  
- Real-time dashboard with auto-refresh every 5 seconds  
- AI-powered failure analysis using Claude  
  
## Tech Stack  
Backend: Python 3.11 · FastAPI · PostgreSQL (Supabase) · Claude AI  
Frontend: React 18 · Tailwind CSS · Recharts  
Hosting: Railway (backend) · Vercel (frontend)  
  
## Live Demo  
https://simulator.yourdomain.com  
  
## Quick Start  
See installation guide in docs/
```

Day 21 — Portfolio Assets

Resume Bullet Points

Customer Integration Simulator | palaemonsystems.com/work

- Built production-grade webhook integration simulator with AI-powered debugging
- Implemented exponential backoff retry engine processing 100+ events/second
- Integrated Claude AI for automated failure analysis and fix recommendations
- Tech: Python, FastAPI, React, PostgreSQL — deployed on Railway + Vercel

LinkedIn Post Template



Just launched: Customer Integration Simulator

Built a production-grade webhook integration testing platform in 3 weeks.

Key features:

- Event-driven architecture with intelligent retry logic
- Real-time observability dashboard
- AI-powered failure analysis using Claude
- Zero ongoing cost — all free tier hosting

Live demo: <https://simulator.yourdomain.com>

[#WebhookIntegrations](#) [#SystemDesign](#) [#Python](#) [#React](#) [#AI](#)

Positioning Statement — Use in Interviews

"A distributed webhook processing simulation platform with failure injection, retry orchestration, real-time observability, and AI-assisted root cause analysis."

✓ Final Day 21 Checklist

Backend

- ☐ All env variables set in Railway
- ☐ GET /health returns healthy on live Railway URL
- ☐ /api/docs accessible publicly
- ☐ CORS includes Vercel URL and custom domain
- ☐ Logs visible in Railway dashboard

Frontend

- ☐ REACT_APP_API_URL points to Railway backend
- ☐ Dashboard loads on Vercel URL
- ☐ Can send test webhooks on live site
- ☐ Event detail modal works
- ☐ AI debugger shows analysis

Domain & SSL

- ☐ Custom domain DNS records added
- ☐ HTTPS active — no browser certificate warning
- ☐ Backend CORS updated with custom domain

Portfolio

- ☐ GitHub repo is public
- ☐ README.md complete
- ☐ Portfolio article written
- ☐ Screenshot of dashboard taken
- ☐ palaemonsystems.com/work updated
- ☐ Resume updated with bullet points
- ☐ LinkedIn post drafted and ready to publish



Project Complete — Go Apply for Jobs!

SUPPLEMENT Missing Files & Critical Fixes

Open this when you hit import errors, CORS issues, or anything unexpected

 Reference throughout all 21 days

What This Fixes

These files were referenced in the main guides but never fully defined. Apply each fix at the checkpoint noted. Skipping will cause import errors, CORS failures, and broken AI endpoints.

#	File	When to Add	Fixes
1	<code>frontend/src/utils/constants.js</code>	Day 12 — before Part 4	Missing <code>INTEGRATION_TYPES</code> , <code>EVENT_TYPES</code> imports
2	<code>frontend/src/utils/formatters.js</code>	Day 12 — before Part 4	Missing <code>formatPercentage</code> , <code>formatTimestamp</code> imports
3	<code>backend/database/repository.py</code>	Day 7 — before Part 5	<code>get_logs_for_event()</code> method needed by AI endpoint
4	<code>backend/app/main.py</code> (CORS)	Day 5 — in Part 2	Frontend can't reach backend without this
5	Local testing workflow	Day 14 — before Part 6	Verify before deploying
6	Supabase RLS SQL	Before going public	Security — not needed for dev

Fix 1 — frontend/src/utils/constants.js

Add on **Day 12**, before starting Part 4 (04-FRONTEND-COMPLETE.docx). Required by SimulatorControls.jsx and StatusBadge.jsx.

```
// frontend/src/utils/constants.js
// CRITICAL: Create this file before starting Part 4

export const EVENT_STATUS = {
  RECEIVED: 'received',
  VALIDATED: 'validated',
  QUEUED: 'queued',
  PROCESSING: 'processing',
  SUCCESS: 'success',
  FAILED: 'failed',
  RETRYING: 'retrying',
  PERMANENT_FAILURE: 'permanent_failure',
  REJECTED: 'rejected',
  TIMED_OUT: 'timed_out',
};

export const STATUS_COLORS = {
  received: 'bg-blue-100 text-blue-800',
  validated: 'bg-blue-100 text-blue-800',
  queued: 'bg-yellow-100 text-yellow-800',
  processing: 'bg-yellow-100 text-yellow-800',
  success: 'bg-green-100 text-green-800',
  failed: 'bg-red-100 text-red-800',
  retrying: 'bg-orange-100 text-orange-800',
  permanent_failure: 'bg-red-100 text-red-800',
  rejected: 'bg-gray-100 text-gray-800',
  timed_out: 'bg-red-100 text-red-800',
};

export const INTEGRATION_TYPES = [
  { value: 'stripe', label: 'Stripe' },
  { value: 'shopify', label: 'Shopify' },
  { value: 'crm', label: 'CRM' },
  { value: 'generic', label: 'Generic' },
];

export const EVENT_TYPES = {
  stripe:
    ['payment.succeeded', 'payment.failed', 'charge.refunded', 'customer.created', 'subscription.updated'],
  shopify:
    ['order.created', 'order.updated', 'order.cancelled', 'product.created', 'product.updated'],
  crm:
    ['contact.created', 'contact.updated', 'deal.won', 'deal.lost', 'task.completed'],
  generic: ['event.triggered', 'action.completed', 'webhook.received'],
};
```

Verify

```
# In browser DevTools console after npm start:
# Should see ZERO import errors mentioning 'constants'
```

Fix 2 — frontend/src/utils/formatters.js

Add on **Day 12**, before starting Part 4. Required by Dashboard.jsx and EventRow.jsx.

```
// frontend/src/utils/formatters.js
// CRITICAL: Create this file before starting Part 4

import { format, formatDistanceToNow } from 'date-fns';

export const formatTimestamp = (ts) => {
  if (!ts) return '-';
  try { return format(new Date(ts), 'MMM dd, yyyy HH:mm:ss'); }
  catch { return ts; }
};

export const formatRelativeTime = (ts) => {
  if (!ts) return '-';
  try { return formatDistanceToNow(new Date(ts), { addSuffix: true }); }
  catch { return ts; }
};

export const formatDuration = (ms) => {
  if (ms == null) return '-';
  return ms < 1000 ? `${ms}ms` : `${(ms/1000).toFixed(2)}s`;
};

export const truncateId = (id, len = 8) => {
  if (!id) return '-';
  const s = String(id);
  return s.length <= len ? s : `${s.slice(0, len)}...`;
};

export const formatPercentage = (val) => {
  if (val == null) return '-';
  return `${Number(val).toFixed(1)}%`;
};

export const formatNumber = (num) => {
  if (num == null) return '-';
  return Number(num).toLocaleString();
};
```

Fix 3 — Log Repository Method

Add on **Day 7** to the existing `LogRepository` class in `backend/database/repository.py`. Required by Part 5 AI endpoint.

```
# Add to the existing LogRepository class in repository.py

def get_logs_for_event(self, event_id: UUID) -> List[Dict[str, Any]]:
    """
    Get all logs for an event ordered by timestamp.
    CRITICAL: This method was missing from the original guide.
    Required by Part 5 AI debug endpoint.
    """
    session = db_manager.get_session()
    try:
        query = text("""
            SELECT log_id, timestamp, level, message,
                   metadata, exception_type, duration_ms
            FROM event_logs
            WHERE event_id = :event_id
            ORDER BY timestamp ASC
            """)
        result = session.execute(query, {'event_id': str(event_id)})
        return [
            {
                'log_id': str(row.log_id),
                'timestamp': row.timestamp.isoformat() if row.timestamp else
None,
                'level': row.level,
                'message': row.message,
                'metadata': row.metadata,
                'exception_type': row.exception_type,
                'duration_ms': row.duration_ms
            }
            for row in result
        ]
    except Exception as e:
        logger.error(f'Error fetching logs for event {event_id}: {e}')
        raise
    finally:
        session.close()
```

Verify

```
cd backend && source venv/bin/activate
python -c "from database.repository import log_repository; print('✓ Log repo
OK')"
```

Fix 4 — CORS Configuration

Add on **Day 5** when building `backend/app/main.py` in Part 2. Without this, every API call from the frontend will fail with a CORS error in the browser console.

```
# backend/app/main.py - complete CORS setup

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        'http://localhost:3000',      # Local development
        'http://localhost:3001',      # Alt local port
        settings.FRONTEND_URL,        # From .env
        'https://your-app.vercel.app', # Update after deploy
        'https://simulator.yourdomain.com', # Update when domain ready
    ],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

# How to test CORS is working:
# Open browser → DevTools → Network tab
# Click 'Send Test Webhook' in frontend
# If CORS is broken: red error, 'has been blocked by CORS policy'
# If CORS works: green 200 response
```


Fix 5 — Local Testing Workflow

Run this on **Day 14** before deploying anything. If a step fails, fix it locally — do not push broken code.

Full Stack Local Test

```
# Step 1 - Start backend (Terminal 1)
cd backend && source venv/bin/activate
python app/main.py
# Wait for: '✓ DB connected' and 'Uvicorn running on http://0.0.0.0:8000'

# Step 2 - Start frontend (Terminal 2)
cd frontend && npm start
# Wait for: 'Compiled successfully!' - browser opens localhost:3000

# Step 3 - API smoke tests (Terminal 3)
curl http://localhost:8000/health
# Expected: {"status":"healthy","database":true}

curl -X POST http://localhost:8000/api/v1/webhook/stripe \
  -H 'Content-Type: application/json' \
  -d '{"event_type":"payment.succeeded","data":{"amount":5000}}'
# Expected: {"event_id":"uuid...", "status":"received"}

curl http://localhost:8000/api/v1/events
# Expected: JSON array with at least 1 event
```

Common Issues & Fixes

Problem	Fix
Frontend can't reach backend (CORS error)	Verify CORS config in main.py includes localhost:3000
Database connection failed	Check DATABASE_URL in .env — include the password
AI analysis fails	Check ANTHROPIC_API_KEY — or use ENVIRONMENT=development for mock mode
npm start fails with import errors	Create constants.js and formatters.js (Fixes 1 & 2 above)
Events don't appear in Supabase	Check Supabase dashboard → Table Editor → events table
Railway deploy fails	Run railway logs — usually a missing env variable or wrong Python version
Vercel frontend blank page	Check REACT_APP_API_URL env var in Vercel dashboard → Settings → Environment Variables

Fix 6 — Supabase Row Level Security (Optional)

Run this in Supabase SQL Editor **before making the project public**. Not needed during development.

```
ALTER TABLE events ENABLE ROW LEVEL SECURITY;
ALTER TABLE event_logs ENABLE ROW LEVEL SECURITY;

-- Allow public read (for dashboard)
CREATE POLICY 'allow_public_read_events'
ON events FOR SELECT USING (true);

CREATE POLICY 'allow_public_read_logs'
ON event_logs FOR SELECT USING (true);

-- Backend uses Supabase service role key for writes — this is automatic
```

Quick Diagnostic Commands

```
# Test Python imports
cd backend && source venv/bin/activate
python -c "from domain.models import Event; print('✓ Models OK')"
python -c "from domain.enums import EventStatus; print('✓ Enums OK')"
python -c "from database.repository import event_repository, log_repository;
print('✓ Repos OK')"
python -c "from services.ai_service import ai_service; print('✓ AI service OK')"

# Test DB connection
python -c "from database.connection import db_manager; print('✓ DB:',
db_manager.test_connection())"

# Test frontend imports (check browser console)
# Open DevTools → Console → look for 'Cannot resolve module' errors
```

Apply fixes at the checkpoints noted — not all at once